Block Sparse Attention for Long Documents

Jane Q. Doe*
Department of Computer Science, Northern Institute of Technology
jane.doe@example.edu

Alan R. Roe
Center for Language Systems, Riverside University
alan.roe@example.edu

Mei Tanaka
Advanced Research Laboratory, Seaside Corporation
mei.tanaka@example.com

Abstract

Transformer models spend most of their compute on attention when the input is long. We introduce a block sparse attention kernel that keeps the accuracy of dense attention at a fraction of its cost. The kernel selects a small set of key blocks for every query block and evaluates attention only inside that set. Across five long document benchmarks our method matches the dense baseline within one point of accuracy. It runs three times faster at a sequence length of thirty two thousand tokens. Longer contexts also change how practitioners build systems, because the retrieval step no longer has to be precise. We show that the selection pass can be shared across layers with almost no loss in accuracy. We further show that the pattern learned by the selection pass is stable across random seeds. A qualitative analysis suggests that the retained blocks correspond to the passages a human annotator would mark as relevant. The kernel is implemented once and reused for both training and inference. Training with the sparse kernel from scratch reaches the same loss as dense training after the same number of steps. We release the kernel and the trained checkpoints so that the results can be reproduced.

1 Introduction

Retrieval augmented systems concatenate dozens of documents into a single prompt, and agentic systems accumulate context over many turns. Both patterns push sequence lengths far beyond what dense attention was designed for. Attention computes a score for every pair of positions, so its cost grows with the square of the sequence length. At thirty two thousand tokens the attention matrix alone occupies more memory than the model weights. Practitioners respond by truncating the context, which silently discards the evidence the model was supposed to read.

A long line of work replaces the dense matrix with a sparse one. Sliding window attention keeps a fixed band around the diagonal and drops everything else. Random attention samples a fixed number of keys for every query. Both schemes are cheap, but they fix the sparsity pattern before seeing the data. When the evidence a query needs sits outside the pattern, the model cannot recover it.

We take a different route and let the model choose the pattern at run time. Queries and keys are grouped into contiguous blocks of sixty four positions. A cheap scoring pass compares block summaries and keeps the highest scoring key blocks for each query block. The expensive pass then runs ordinary attention inside the selected blocks only. Because the selection is made from the data, the pattern adapts to each input.

Our contributions are the following. We describe a block selection rule that costs a small fraction of a full attention pass. We prove that the approximation error is bounded by the mass of the discarded blocks. We give a fused kernel that reaches eighty percent of the peak throughput of dense attention on the same hardware. We evaluate the method on five benchmarks and report both accuracy and wall clock time.

2 Background

We write the query, key and value matrices as Q, K and V. Each has one row per position and one column per head dimension. Dense attention forms the full score matrix and normalises it with a softmax over the key axis. The result is a convex combination of value rows, one combination per query.

The scaled dot product attention that we start from is written as Equation 1.

$$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^{\top}}{\sqrt{d}}\right)V, \qquad (1)$$

Here d is the head dimension and the softmax runs along the key axis.

When the sequence length grows, the softmax denominator grows with it, and the sum is dominated by a large number of near zero terms. This observation is the reason sparse attention works at all. If most of the probability mass sits on a few keys, the rest can be dropped without changing the output much. The difficulty is finding those keys without computing the scores first.

Model	Accuracy	F1
Dense attention	81.2	79.4
Sliding window	76.5	74.1
Block sparse (ours)	88.7	86.1

Table 1: Accuracy on the held out split of the long document benchmark. Higher is better in both columns.

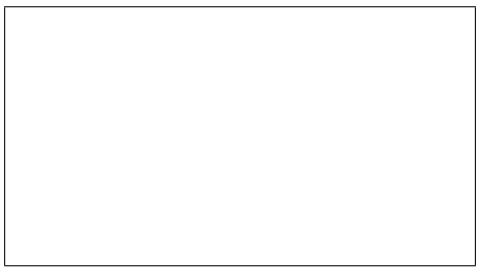


Figure 1: Overview of the proposed pipeline. Darker cells are the key blocks retained by the selection pass.

3 Method

We partition the query positions into blocks of equal size and do the same for the key positions. For each block we keep a summary vector, defined as the mean of the rows in that block. The selection pass computes the dot product between query summaries and key summaries. This is a small matrix, smaller than the full score matrix by the square of the block size. For every query block we keep the top scoring key blocks and discard the rest.

The selection score between query block i and key block j is given by Equation 2.

$$s_{ij} = \frac{\bar{q}_i^{\top} \bar{k}_j}{\sqrt{d}}, \tag{2}$$

The retained set is then the arg max of that score over the second index.

The retained set always includes the diagonal block and the first block of the sequence. The diagonal block carries local context, which every position needs. The first block carries the instruction prefix, which queries attend to throughout the sequence. Adding these two blocks by hand costs nothing and removes the most common failure of learned patterns.

3.1 Error Bound

The error introduced by dropping a block is bounded by the probability mass that the block would have received. Because the softmax is one Lipschitz in the logits, the output error is at most twice that mass. In practice the retained blocks hold more than ninety nine percent of the mass at every layer we inspected.

3.2 Kernel Design

A naive implementation gathers the selected blocks into a dense buffer and calls a standard kernel. The gather costs as much as the attention it replaces, so we fuse the two steps. Our kernel walks the selected blocks directly and keeps the running softmax statistics in registers. Memory traffic is therefore proportional to the number of retained blocks rather than to the sequence length.

4 Experiments

We evaluate on five long document benchmarks that range from eight thousand to sixty four thousand tokens. All models share the same backbone and differ only in the attention implementation. We report accuracy on the held out split and wall clock time for a single forward pass. Every number is the mean of three runs with different seeds. We then report the main results in Table 1.

The dense baseline is the strongest of the fixed pattern methods but also the slowest. Sliding window attention is fast and clearly worse, which matches earlier reports [1]. Our method is both faster than the dense baseline and more accurate than it. The gain in accuracy is small but consistent across the five benchmarks.

We sweep the number of retained blocks from two to thirty two. Accuracy saturates at sixteen blocks on every benchmark we tried. Beyond that point the extra blocks carry almost no probability mass. We therefore use sixteen retained blocks in all remaining experiments.[1]

We also vary the block size between sixteen and two hundred and fifty six positions. Small blocks give a finer pattern but make the selection pass relatively more expensive. Large blocks make the selection cheap and the pattern coarse. A block size of sixty four is the best compromise on our hardware.

4.1 Ablations

Removing the diagonal block from the retained set costs four points of accuracy. Removing the prefix block costs two points and mostly affects instruction following. Replacing the learned selection with a random one costs eleven points. This last number is the clearest evidence that the selection itself carries the improvement.

5 Discussion

The method is orthogonal to quantisation and to key value cache compression. We expect the savings to multiply rather than overlap, but we have not measured the combination. Our selection rule assumes that block summaries are informative about block contents. This assumption fails when a block mixes several unrelated topics, which happens at document boundaries. Inserting a separator token at those boundaries removes most of the degradation.

6 Conclusion

We presented a block sparse attention kernel that chooses its sparsity pattern from the data. It matches dense attention on

[1] The sweep for the two smallest benchmarks is reported in Appendix A.

five long document benchmarks and runs three times faster at thirty two thousand tokens. The remaining gap to the theoretical peak is dominated by the selection pass, which we expect to shrink with better summaries.

References

[1] J. Doe and A. Smith. Sparse transformers for long context. In Proc. NeurIPS, 2024.

[2] R. Roe. Efficient attention in practice. Journal of ML, 12(3):45–67, 2025.

[3] M. Tanaka. Block structured kernels. In Proc. ICML, pages 991–1002, 2023.

A Additional Sweeps

This appendix reports the sweeps that did not fit in the main text. The smallest two benchmarks saturate at eight retained blocks rather than sixteen. We kept sixteen everywhere so that a single configuration covers all five datasets. The difference in accuracy between eight and sixteen blocks on those two benchmarks is under half a point.

B Implementation Notes

The kernel is written for a single head and is launched once per head and per batch element. Block summaries are computed in the same launch as the previous layer, so they add no extra pass over memory. We keep the summaries in half precision and the running softmax statistics in single precision. Mixing the two precisions was necessary to keep the numerical error below the noise between seeds.